МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №1
по курсу “Компьютерная графика”

Основы 3D графики

Выполнил: Д. И. Шнайдер
Группа: М8О-308БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель работы: познакомиться с основами 3D-графики - построением простых трёхмерных объектов, проекцией на двумерную плоскость, матрицами перспективной и ортографической проекции и аффинными преобразованиями. Программа должна работать в реальном времени, использовать Vulkan, GLFW и ImGui и позволять динамически менять проекцию и трансформации объекта.

Вариант 3 - параллелепипед. Номер варианта: ((27 − 1) mod 12) + 1 = 3.

Выполнены задание варианта и пять первых дополнительных заданий:

1. переключение матрицы проекции (ортографическая и перспективная) в интерфейсе;
2. элементы интерфейса для изменения позиции, поворота и растяжения фигуры;
3. движение и поворот фигуры по сложной траектории, кнопка паузы и воспроизведения анимации, изменение скорости анимации и параметров траектории (радиус, высота, частота и др.);
4. изменение цвета фигуры элементом ColorEdit, цвет передаётся в шейдеры через uniform-буфер;
5. процедурные цвета вершин в зависимости от их координат в локальной системе фигуры; итоговый цвет равен произведению цвета вершины и цвета, выбранного в интерфейсе.

Метод решения

Работа выполнена на основе стартового проекта курса (Vulkan, GLFW, ImGui, vk-bootstrap, Vulkan Memory Allocator, сборка CMake). Код работы находится в файлах `application.hpp` и `application.cpp`; векторы и матрицы реализованы в собственном заголовочном файле `math3d.hpp` без сторонних математических библиотек; шейдеры `parallelepiped.vert` и `parallelepiped.frag` компилируются в SPIR-V утилитой glslc при сборке.

Геометрия параллелепипеда

Параллелепипед задаётся тремя векторами рёбер **a**, **b**, **c**, выходящими из одной вершины. Центр фигуры совмещён с началом локальной системы координат, тогда восемь вершин:

$$\mathbf{P}_{ijk} = \left(i - \frac{1}{2}\right)\mathbf{a} + \left(j - \frac{1}{2}\right)\mathbf{b} + \left(k - \frac{1}{2}\right)\mathbf{c}, \;\; i,j,k \in \{0,1\}.$$

Векторы рёбер вычисляются по длинам a, b, c и углам между рёбрами α (между **b** и **c**), β (между **a** и **c**), γ (между **a** и **b**): ребро **a** лежит на оси X, ребро **b** - в плоскости XY, направление **c** находится из условий на скалярные произведения:

$$\mathbf{a} = a(1,0,0), \;\; \mathbf{b} = b(\cos\gamma, \sin\gamma, 0), \;\; \mathbf{c} = c\left(c_x, c_y, c_z\right),$$

$$c_x = \cos\beta, \;\; c_y = \frac{\cos\alpha - \cos\beta\cos\gamma}{\sin\gamma}, \;\; c_z = \sqrt{1 - {c_x}^2 - {c_y}^2}.$$

Если подкоренное выражение неположительно, такая тройка углов не может сойтись в одной вершине, и интерфейс выводит предупреждение. При $\alpha = \beta = \gamma = 90°$ получается прямоугольный параллелепипед (по умолчанию 2 × 1,2 × 0,8). Длины рёбер и углы меняются ползунками в разделе «Фигура», после чего вершинный буфер заполняется заново.

Фигура хранится как **24 вершины и 36 индексов**: по 4 собственные вершины на каждую из 6 граней и по 2 треугольника (6 индексов) на грань. Угловая вершина принадлежит трём граням с разными нормалями, поэтому вершины граней не объединяются в 8 общих: это позволяет хранить нормаль грани в каждой вершине (используется для затенения и пригодится в работах с освещением). Индексы имеют тип uint16.

Нормаль грани, натянутой на рёбра **u** и **w**, равна

$$\mathbf{n} = \frac{\mathbf{u} \times \mathbf{w}}{|\mathbf{u} \times \mathbf{w}|}.$$

Если **n** направлена внутрь фигуры (её скалярное произведение с вектором из центра в центр грани отрицательно), векторы **u** и **w** меняются местами. Это одновременно разворачивает нормаль наружу и делает обход вершин каждого треугольника направленным против часовой стрелки при взгляде снаружи, поэтому в конвейере включено отсечение задних граней. Цвет вершины вычисляется по её положению (см. ниже). Структура вершины `Vertex` содержит позицию, нормаль и цвет - по три float, шаг 36 байт, формат атрибутов `VK_FORMAT_R32G32B32_SFLOAT`.

Матрицы преобразований

Используются однородные координаты и матрицы 4 × 4, которые хранятся по столбцам - так же, как `mat4` в GLSL, поэтому матрица копируется в uniform-буфер без перестановки. Вершина переводится в пространство отсечения произведением

$$\mathbf{p}_{\text{clip}} = \mathbf{P} \cdot \mathbf{V} \cdot \mathbf{M} \cdot \mathbf{p},$$

где **M** - матрица модели, **V** - матрица вида, **P** - матрица проекции. Матрица модели собирается из аффинных преобразований:

$$\mathbf{M} = \mathbf{T}\big(\mathbf{p}_0 + \mathbf{s}(\tau)\big) \cdot \mathbf{R}_z\mathbf{R}_y\mathbf{R}_x \cdot \mathbf{R}_{\text{ось}}(\omega_c\tau) \cdot \mathbf{S}\big(k_x, k_y, k_z\big),$$

где $\mathbf{p}_0$ - позиция из интерфейса, $\mathbf{s}(\tau)$ - смещение по траектории анимации, $\mathbf{R}_z\mathbf{R}_y\mathbf{R}_x$ - поворот углами Эйлера из интерфейса, $\mathbf{R}_{\text{ось}}$ - вращение вокруг собственной оси фигуры, **S** - растяжение по осям. Порядок справа налево: сначала растяжение, затем вращение вокруг своей оси, поворот и перенос.

Камера движется по сфере радиуса d вокруг начала координат (азимут θ, наклон ψ): $\mathbf{e} = d(\cos\psi \sin\theta, \sin\psi, \cos\psi \cos\theta)$. Матрица вида строится функцией lookAt: её строки - базисные векторы камеры (вправо, вверх, назад), последний столбец переносит камеру в начало координат.

В Vulkan ось Y нормализованных координат направлена вниз, а глубина лежит в отрезке [0; 1], поэтому матрицы проекции составлены под эти соглашения. Перспективная проекция (φ - вертикальный угол обзора, A - соотношение сторон окна, n, f - ближняя и дальняя плоскости, $g = \operatorname{ctg}(\varphi/2)$):

$$\mathbf{P}_{\text{персп}} = \begin{bmatrix} \frac{g}{A} & 0 & 0 & 0 \\ 0 & -g & 0 & 0 \\ 0 & 0 & \frac{f}{n-f} & \frac{nf}{n-f} \\ 0 & 0 & -1 & 0 \end{bmatrix}.$$

Знак минус у элемента $-g$ переворачивает ось Y, третья строка переводит точку с $z = -n$ в глубину 0, а точку с $z = -f$ - в глубину 1, четвёртая строка даёт $w = -z$, и после деления на w удалённые объекты становятся меньше. Ортографическая проекция видимого объёма высотой H и шириной $W = AH$:

$$\mathbf{P}_{\text{орто}} = \begin{bmatrix} \frac{2}{W} & 0 & 0 & 0 \\ 0 & -\frac{2}{H} & 0 & 0 \\ 0 & 0 & -\frac{1}{f-n} & -\frac{n}{f-n} \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Здесь $w = 1$, деления на глубину нет: параллельные рёбра остаются параллельными, размер фигуры не зависит от расстояния до камеры. Кнопка «Как у перспективы» выбирает $H = 2d\,\mathrm{tg}(\varphi/2)$, чтобы при переключении проекций размер фигуры в центре сцены совпадал. Тип проекции переключается радиокнопками (дополнительное задание 1).

Для преобразования нормалей в шейдер передаётся матрица $\mathbf{N} = (\mathbf{RS})^{-\mathrm{T}} = \mathbf{RS}^{-1}$ ($\mathbf{R}$ ортогональна): при неравномерном растяжении нормали остаются перпендикулярными граням.

Анимация по траектории

Анимационное время накапливается каждый кадр: $\tau \leftarrow \tau + \Delta t \cdot k$, где Δt - время кадра, k - множитель скорости из интерфейса. Пауза останавливает накопление, поэтому фигура замирает на месте и продолжает движение с той же точки; скорость не зависит от частоты кадров. Угол на траектории $\vartheta = \omega\tau$. Реализованы три траектории (радиус R, высота h, число волн за оборот m, число витков за подъём q):

- окружность с волнами: $\mathbf{s} = (R\cos\vartheta, h\sin m\vartheta, R\sin\vartheta)$;
- спираль вверх-вниз: $\mathbf{s} = (R\cos\vartheta, h(2\lambda - 1), R\sin\vartheta)$, где λ линейно растёт от 0 до 1 за q витков, затем убывает;
- восьмёрка (лемниската Жероно): $\mathbf{s} = (R\sin\vartheta, h\sin m\vartheta, R\sin\vartheta\cos\vartheta)$.

Одновременно фигура вращается вокруг собственной оси X, Y или Z с угловой скоростью ω_c. В разделе «3. Анимация» есть кнопки «Пауза/Воспроизвести» и «В начало», выбор траектории и ползунки для всех её параметров (дополнительное задание 3). Позиция, повороты и растяжение задаются элементами DragFloat3 и SliderFloat3 (дополнительное задание 2); растяжение ограничено снизу значением 0,05, так как нулевой или отрицательный масштаб вырождает матрицу нормалей и меняет направление обхода треугольников.

Цвет фигуры

Процедурный цвет вершины (дополнительное задание 5) задаётся её положением в локальной системе координат: каждая координата p_i нормируется по габаритам фигуры (e_i - половина габаритного размера по оси) и становится компонентой цвета (x → красный, y → зелёный, z → синий):

$$c_i = 0{,}15 + 0{,}85 \cdot \frac{p_i + e_i}{2e_i}, \quad i \in \{x, y, z\}.$$

Нижняя граница 0,15 нужна, чтобы угол с минимальными координатами не был чёрным. Цвет из элемента ColorEdit3 (дополнительное задание 4) передаётся во фрагментный шейдер через uniform-буфер, итоговый цвет фрагмента - покомпонентное произведение:

$$\mathbf{c}_{\text{итог}} = \mathbf{c}_{\text{вершины}} \odot \mathbf{c}_{\text{UI}}.$$

Флажок «Цвета вершин по координатам» отключает множитель цвета вершины (тогда итоговый цвет равен цвету из интерфейса). Флажок «Затенение граней по нормалям» дополнительно умножает цвет на 0,35 + 0,65 · max(n · l, 0), где l - фиксированное направление на источник света: при однотонной заливке иначе грани сливаются в один силуэт.

Объекты Vulkan и отрисовка

Стартовый код создаёт экземпляр, устройство, swapchain, проход рендеринга с вложениями цвета и глубины, кадровые буферы, командный буфер и синхронизацию кадра. В работе добавлены:

- три объекта `VkBuffer` - вершинный, индексный и uniform; память выделяется через VMA в видимой процессору памяти и остаётся отображённой (`VMA_ALLOCATION_CREATE_MAPPED_BIT`), после записи вызывается `vmaFlushAllocation`;
- `VkDescriptorSetLayout` с одной привязкой (binding = 0, `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER`, вершинная и фрагментная стадии), `VkDescriptorPool` и `VkDescriptorSet`, связанный с uniform-буфером через `vkUpdateDescriptorSets`;
- `VkShaderModule` для вершинного и фрагментного шейдеров (SPIR-V) и `VkPipelineLayout`;
- графический конвейер `VkPipeline`: список треугольников, описание вершинных атрибутов (location 0–2), отсечение задних граней (`VK_CULL_MODE_BACK_BIT`, обход против часовой стрелки), тест и запись глубины (`VK_COMPARE_OP_LESS`), динамические

viewport и scissor, чтобы окно можно было менять в размере без пересоздания конвейера.

Каждый кадр функция `update()` обрабатывает интерфейс и вычисляет матрицы, а `render()` записывает данные в uniform-буфер и формирует командный буфер: `vkCmdBeginRenderPass` (очистка цвета и глубины значением 1,0), `vkCmdSetViewport` и `vkCmdSetScissor`, `vkCmdBindPipeline`, `vkCmdBindDescriptorSets`, `vkCmdBindVertexBuffers`, `vkCmdBindIndexBuffer`, `vkCmdDrawIndexed` (36 индексов), `vkCmdEndRenderPass`. Запись в uniform- и вершинный буферы выполняется в `render()`, а не в `update()`: в стартовом коде один кадр «в полёте», и только после ожидания fence в `prepare()` гарантировано, что GPU закончил читать данные предыдущего кадра.

Uniform-блок объявлен в шейдерах с раскладкой std140, и структура `UniformData` в C++ повторяет её байт в байт (смещения проверяются `static_assert`). Тип `mat3` не используется, потому что в std140 каждый его столбец выравнивается до 16 байт; цвет хранится как `vec4`, а не `vec3`, по той же причине:

Поле	Тип GLSL	Смещение, байт	Размер, байт
`mvp`	mat4	0	64
`normal_matrix`	mat4	64	64
`color`	vec4	128	16
`use_vertex_colors`	uint	144	4
`use_lighting`	uint	148	4
(выравнивание)	—	152	8
Итого			160

Таблица 1 — Раскладка uniform-буфера (std140)

Кроме того, в стартовом коде (`graphics_internal.cpp`) увеличен размер пула дескрипторов ImGui: число наборов было равно числу изображений swapchain, а ImGui 1.92 требует минимум три набора (два сэмплера и атлас шрифта). При swapchain из двух изображений программа завершалась с ошибкой ещё до отрисовки первого кадра. Для подписей интерфейса на русском языке загружается системный шрифт Segoe UI.

Результаты

Слева в окне программы находится панель ImGui, разделы которой соответствуют заданиям: «Фигура» (вариант), «1. Проекция и камера», «2. Положение, поворот, растяжение», «3. Анимация», «4-5. Цвет». Фигура отрисовывается в реальном времени, все параметры применяются сразу.

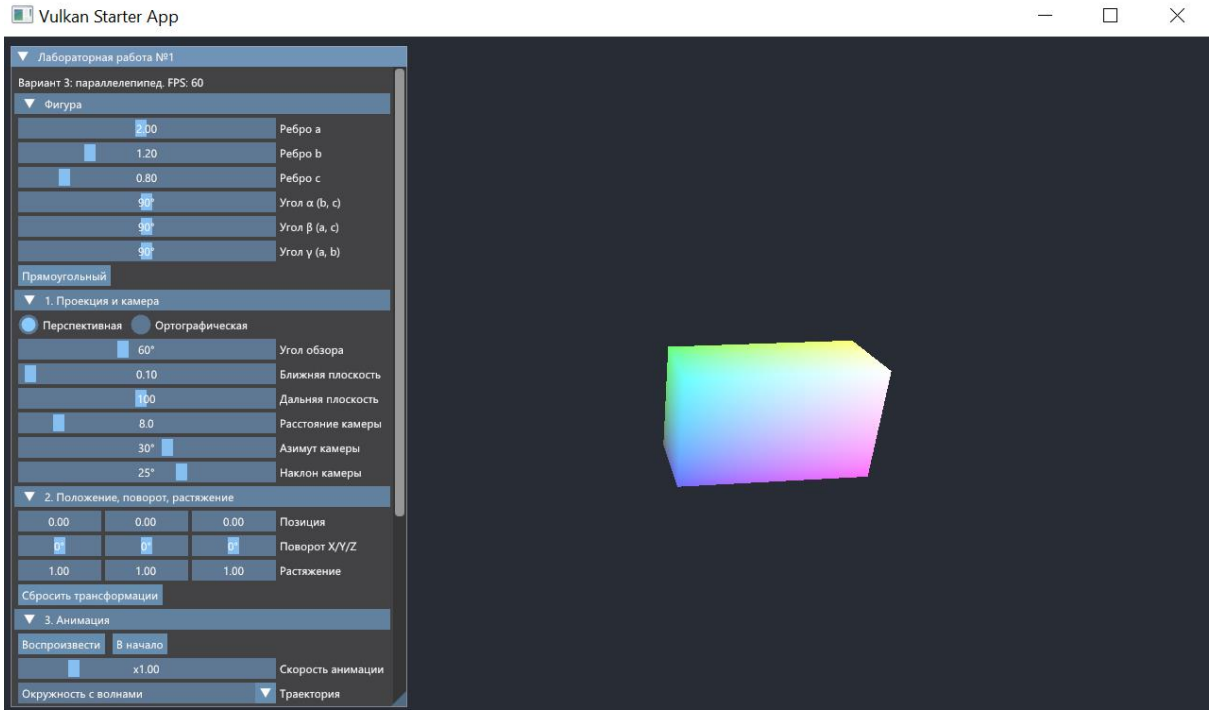


Рисунок 1 - Прямоугольный параллелепипед в перспективной проекции, процедурные цвета вершин

На рисунке 1 параллелепипед с рёбрами 2 × 1,2 × 0,8 показан в перспективной проекции (угол обзора 60°). Цвет из интерфейса белый, поэтому видны процедурные цвета вершин: красная компонента растёт вдоль оси X, зелёная — вдоль Y, синяя — вдоль Z; внутри граней цвета плавно интерполируются. Видны три грани, задние грани отсечены, перекрытия обрабатываются буфером глубины.

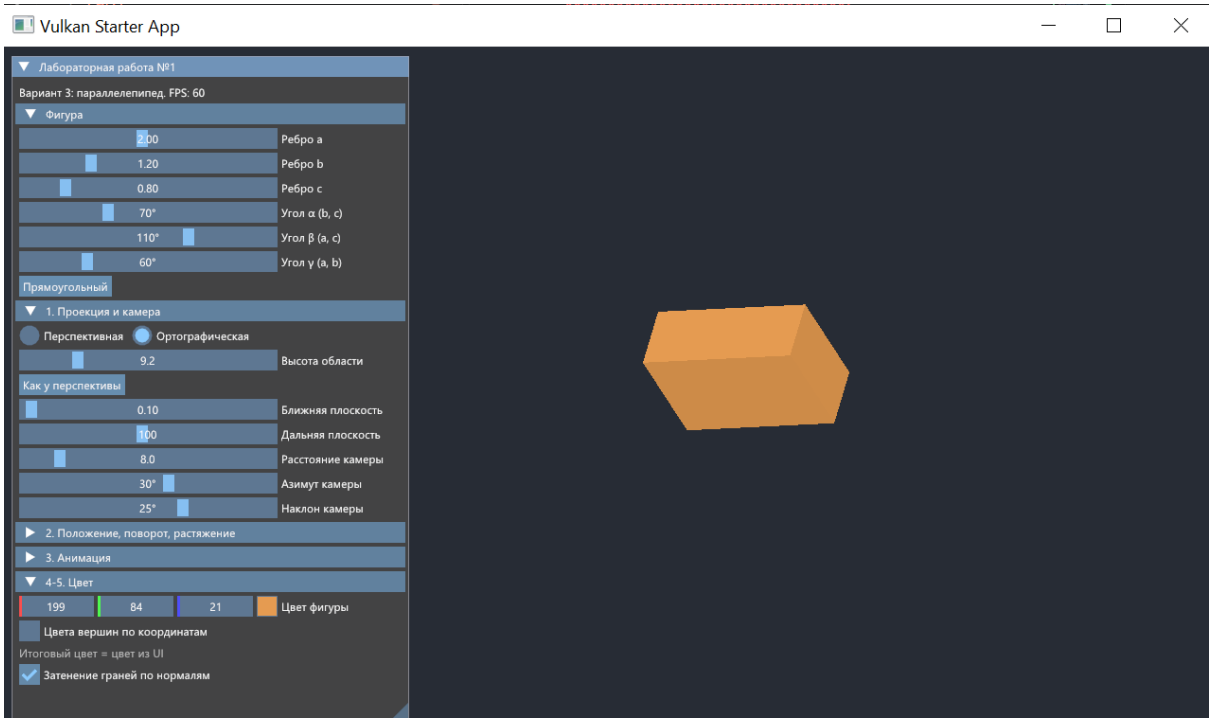


Рисунок 2 - Наклонный параллелепипед в ортографической проекции, цвет из ColorEdit

На рисунке 2 включена ортографическая проекция, углы между рёбрами изменены (α = 70°, β = 110°, γ = 60°), цвет фигуры выбран в ColorEdit3, процедурные цвета отключены и включено затенение граней по нормалям. Параллельные рёбра на экране остаются параллельными, размер фигуры не зависит от расстояния до камеры.

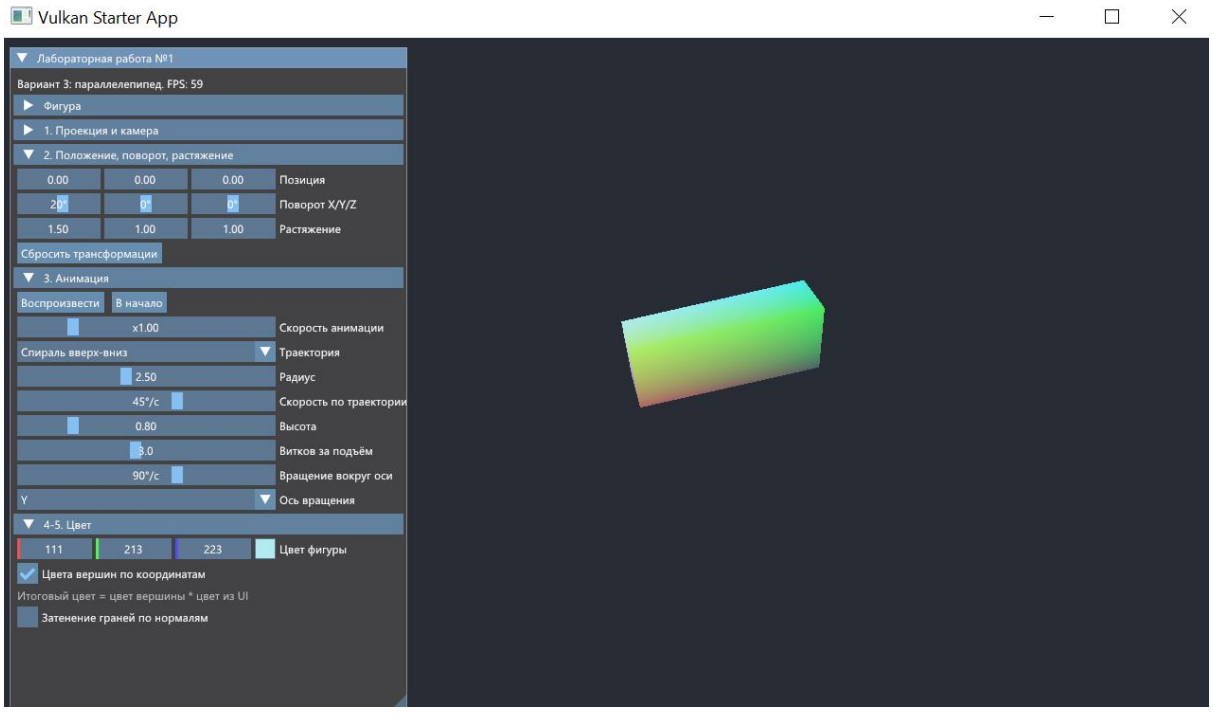


Рисунок 3 - Анимация по траектории и пользовательские трансформации

На рисунке 3 фигура движется по траектории (спираль вверх-вниз или восьмёрка) и одновременно вращается вокруг собственной оси; в разделе «2» заданы растяжение и поворот. Итоговый цвет равен произведению процедурного цвета вершины и цвета из интерфейса. Кнопка «Пауза» останавливает фигуру в текущем положении, ползунки меняют скорость, радиус, высоту и частоту траектории прямо во время движения.

Выводы

В ходе работы я научилась строить трёхмерную фигуру из вершинного и индексного буферов и задавать параллелепипед векторами рёбер, вычисленными по длинам и углам; составлять матрицы модели, вида, перспективной и ортографической проекции с учётом соглашений Vulkan (ось Y направлена вниз, глубина в отрезке [0; 1]) и проверять их на контрольных точках.

Я разобралась, как данные попадают в шейдеры: вершинные атрибуты описываются в конвейере, а матрицы и цвет передаются через uniform-буфер и набор дескрипторов; при этом раскладка структуры в C++ должна совпадать с правилами std140. Также я научилась создавать графический конвейер с тестом глубины и отсечением задних граней, понимать, почему для этого важен порядок обхода вершин и как на него влияет переворот оси Y в матрице проекции, обновлять буферы только после того, как GPU закончил предыдущий кадр, и управлять параметрами сцены и анимацией через ImGui.

Сравнение проекций показало, что перспективная проекция передаёт глубину за счёт уменьшения удалённых частей, а ортографическая сохраняет параллельность и размеры и удобна для оценки формы, например углов наклонного параллелепипеда.